Deep Learning and Computation

Building and training a deep network — Lecture 22 (Chapter 10, Part 3)

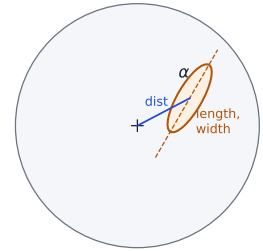
Amir Farbin

Part 3 — Building and training a deep network

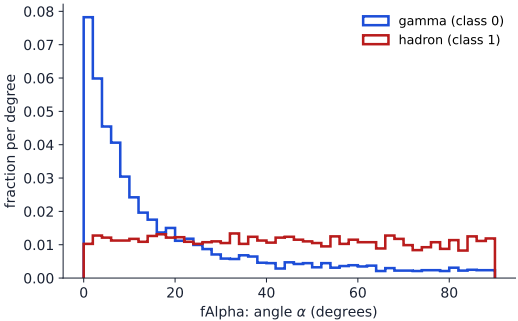
Where we are

- Lecture 21: a first network in Keras, on SUSY. Today: the **same template on a real table**, and the choices behind each line
- The table: one of the benchmark's datasets from Lecture 20, loaded through a **module** (`dataloader.py`): a `.py` file of functions you `import`
- Why a module: freeze finished exploration into one version, so every notebook gets the same thing. A result must be **reproducible**: a clean top-to-bottom run gives it again

The data: the MAGIC gamma-ray telescope

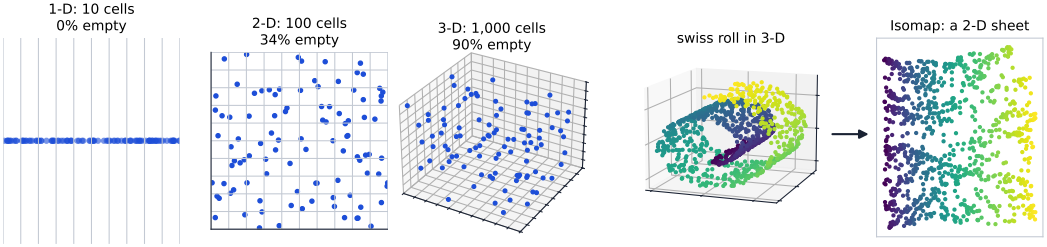


camera; + marks where the telescope points



- A gamma ray or a cosmic-ray **hadron** hits the atmosphere: an **air shower** of particles, which emit faint **Cherenkov light**
- A telescope photographs it: an ellipse. Its shape (**Hillas parameters**) are the 10 features. Class 0 = gamma, 1 = hadron
- Gamma showers point back at the source, so their ellipse points at the camera centre: small α

Natural data lives on manifolds

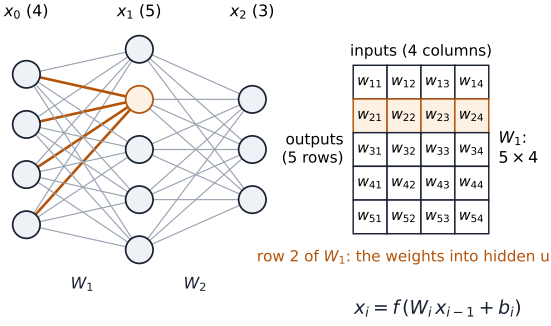


- The same points fill one dimension and leave almost every box empty in three: the **curse of dimensionality** (data grows sparse as dimensions are added)
- A 100×100 photo is a point in 10,000 dimensions, but all photos of one face form a thin, curved sheet, a **manifold**. Moving along it is turning the head, or smiling

Why deep: a hierarchy of features

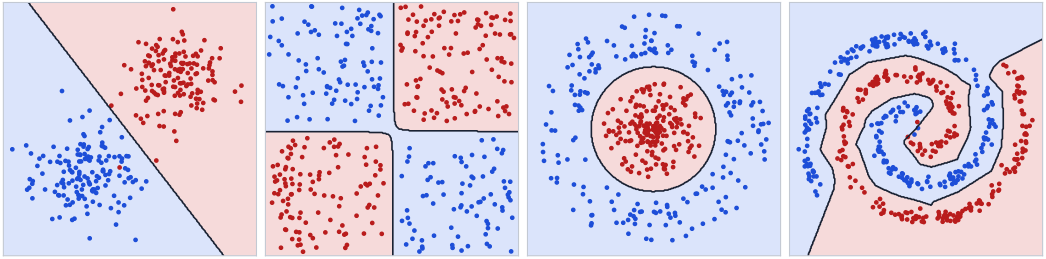
- Pixels $\rightarrow$ edges $\rightarrow$ parts $\rightarrow$ objects. Characters $\rightarrow$ words $\rightarrow$ sentences
- Low layers are **shared** by every category; high layers are **specific** and invariant
- A shallow network can do it in principle, but needs far more neurons

A dense network



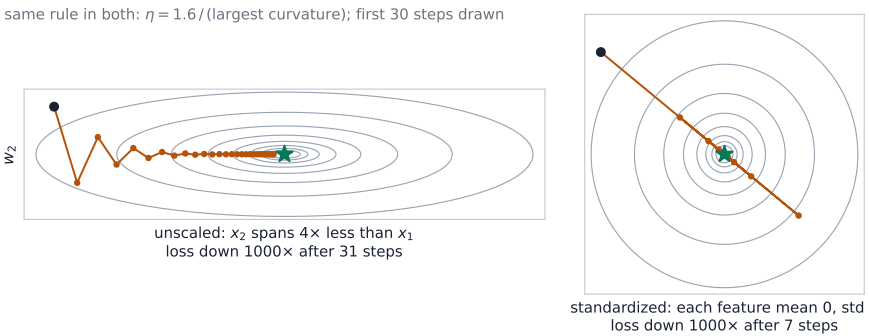
- Layer i: $x_i = f(W_i x_{i-1} + b_i)$, with W_i of shape $n_i \times n_{i-1}$. The network is a composition of layers
- Depth and widths are **hyper-parameters**: settings you choose, not weights training learns. Don't make any layer narrower than the information it must carry

Four datasets, four lessons



- Linear: one sigmoid neuron is **logistic regression**. XOR: add x_1x_2. Circle: add $x_1^2 + x_2^2$: **feature engineering**, building a feature by hand
- The spiral needs a real network, and with a **linear** activation no amount of depth helps

Scale your inputs



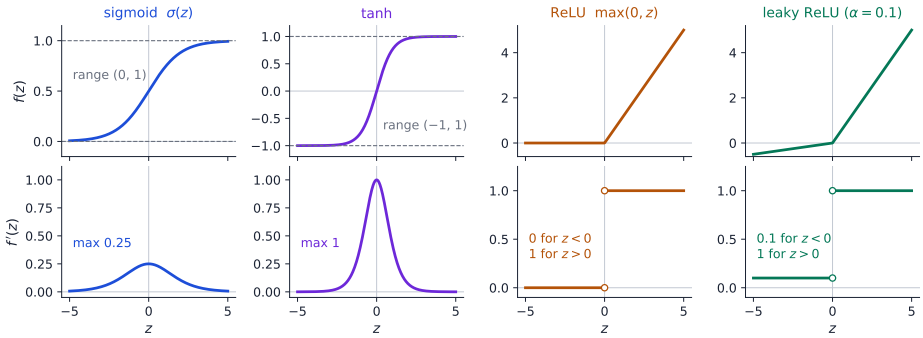
- Standardize: $(x - \text{mean})/\text{standard deviation}$ (`StandardScaler`), or min-max to $[0, 1]$ (`MinMaxScaler`)
- Fit the scaler on the **training** data only

Scaling inside the model: `Normalization`

```
normalizer = keras.layers.Normalization(axis=-1)
normalizer.adapt(X_Train)      # = fit: mean, variance
x = normalizer(in_x)           # = transform, in the model
x = Dense(12, activation='relu')(x)
```

- MAGIC's features run from 0–0.9 to about ±500. Unscaled, epoch 1's loss was **3.16**; normalized, **0.53**
- `summary()`: 21 **non-trainable** parameters (10 means, 10 variances, a count). The scaling travels with the model
- Wire it in: the first `Dense` must be called on the normalizer's **output**, not on `in_x`

Activations inside the network

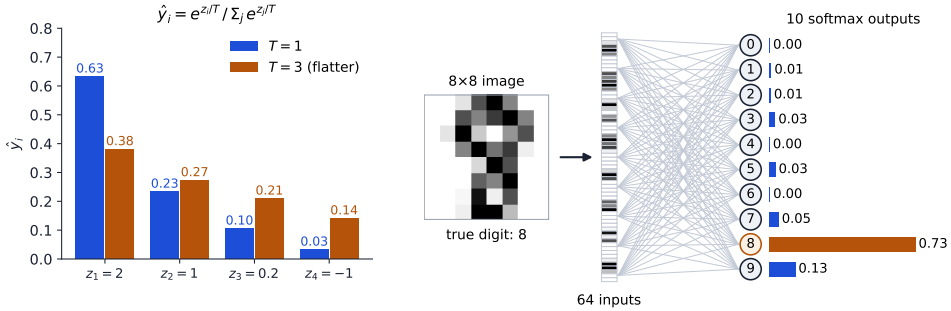


- Sigmoid $(0, 1)$ and tanh $(-1, 1)$ **saturate** (flatten out for large inputs): slope ≈ 0, so gradients vanish
- ReLU: slope **1 or 0**, and cheap. Leaky ReLU avoids units that never switch on

The output layer is the task

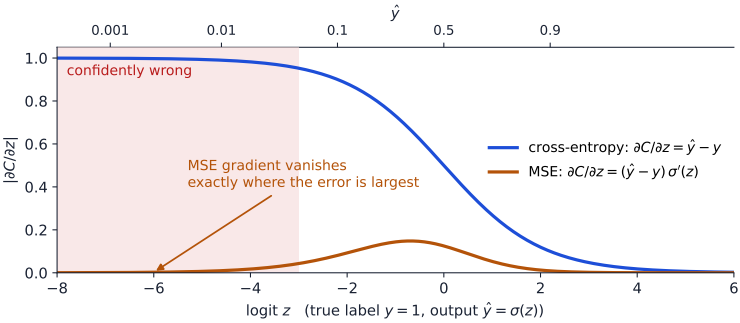
- **Two classes:** one sigmoid output, **binary cross-entropy**
- K **classes:** K softmax outputs, **categorical cross-entropy**, **one-hot** y (a 1 for the right class, 0 elsewhere); integer labels: `sparse_categorical_crossentropy`
- **Regression:** linear output, **mean squared error**
- Everything else is a choice you can tune. This one you must get right

Softmax



- $\mathrm{softmax}(z)_i = e^{z_i} / \sum_j e^{z_j}$: positive, sums to 1
- The Boltzmann factor with $z_i = -E_i/kT$. A language model's "temperature" is this T

Cost functions are likelihoods



- The **likelihood** is the probability of the data given the model. MSE gives the conditional **mean** (a Gaussian likelihood); absolute error gives the **median**
- MSE on a sigmoid: when confidently wrong, the gradient vanishes. Cross-entropy: $\partial C/\partial z = \hat{y} - y$

Cross-entropy and KL divergence

class i	**cat**	**dog**	**bird**
truth y_i (one-hot)	0	1	0
prediction $\hat{y}_i$	0.2	0.7	0.1
$-y_i \log \hat{y}_i$	0	$-\log 0.7 = 0.36$	0

$$C = -\sum_i y_i \log \hat{y}_i = -\log \hat{y}_{\text{dog}} = -\log 0.7 \approx 0.36$$

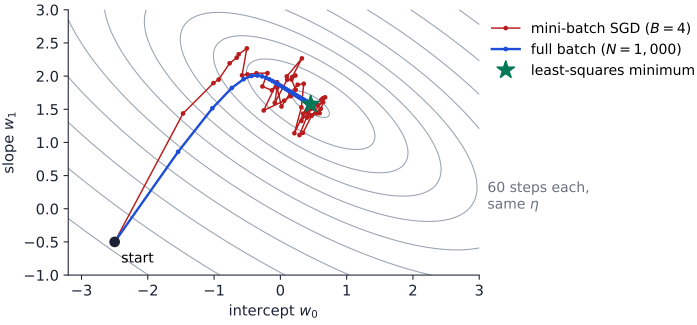
truth outside the log, prediction inside; natural log (nats)

- $C = -\sum_i y_i \log \hat{y}_i$: the truth outside the log, the prediction inside
- **KL divergence** $D_{KL}(p\|q) = \sum p \log(p/q)$: how far q is from p; zero when equal. Since $H(p, q) = H(p) + D_{KL}(p\|q)$, minimizing cross-entropy minimizes KL

Training is walking downhill, and the slope is free

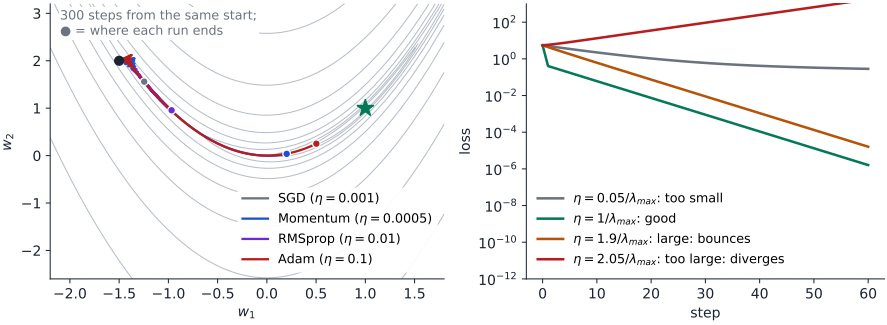
- $\theta \leftarrow \theta - \epsilon \nabla C(\theta)$: parameters θ, **learning rate** ϵ, gradient of the cost ∇C
- Finite differences cost **two evaluations per parameter**: hopeless for millions of parameters
- **Automatic differentiation** (backpropagation) gives the whole gradient for about the cost of one evaluation
- The framework does it for you

Mini-batches, batches, epochs



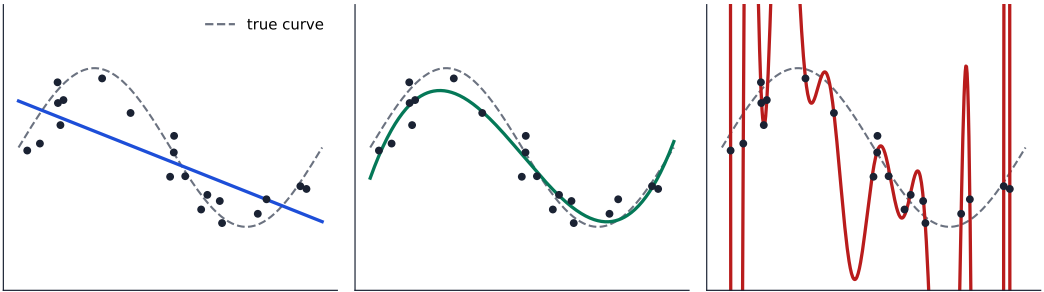
- N = batch size $\times$ number of batches. One **epoch** = one pass through the data
- Many cheap, noisy steps beat a few exact ones, and the noise tends to help

Learning rate, momentum, Adam



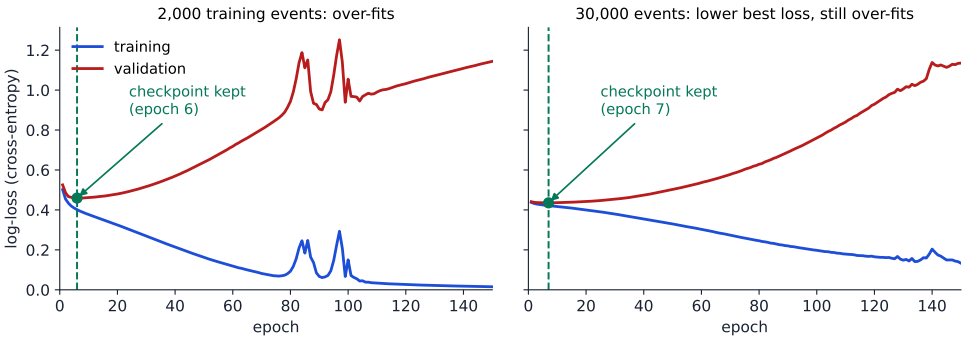
- The learning rate is probably the **most important** hyper-parameter. A **schedule** lowers it as training goes on
- **Momentum** keeps a running average of gradients. **Adam** (and AdamW) adds per-parameter scaling: the default

Under-fitting and over-fitting



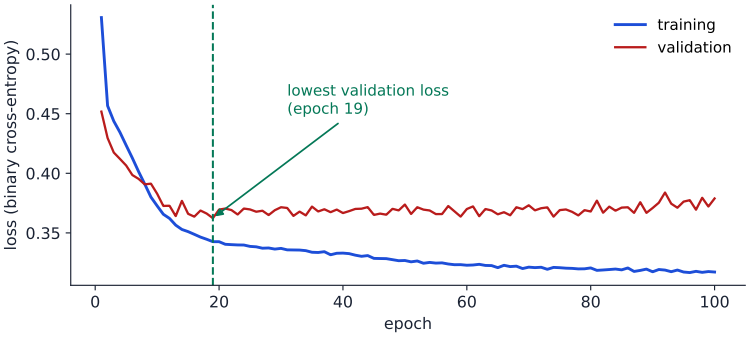
- Big networks can **memorize**. Compare with a simple baseline
- 100% accuracy usually means the answer leaked into the inputs

Learning curves and early stopping



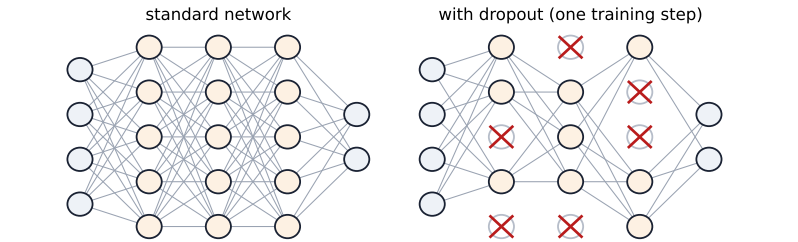
- Training and **validation** loss per epoch: the opening gap is the sign of over-fitting
- Keep the checkpoint from before they separate (`EarlyStopping(restore_best_weights=True)`). The test set stays untouched

Our MAGIC run over-trains



- 100 epochs, batch 10: validation is best at epoch 19, training keeps improving. The gap is memorization
- Validation wiggles more: 802 events against 6,420. Stop near the minimum, or keep the best checkpoint

Regularization: L2 and dropout

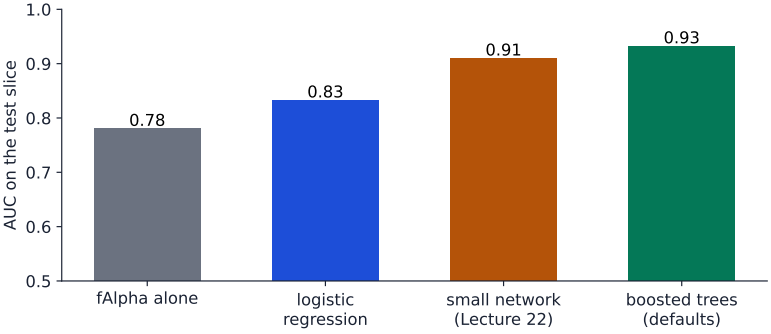


training only; Keras Dropout(rate) = fraction dropped (here 0.4); kept outputs scaled by $1/(1-\text{rate})$

dropped units output 0: their activations are zeroed, not their weights

- **L2:** add $\lambda \sum w^2$ to the cost (L1: $\lambda \sum |w|$): small weights, smoother functions
- **Dropout** (2012): zero a random fraction of unit **outputs** while training; off when predicting

How good is that? Climb the baseline ladder



- Same training and test slices. One feature, then a linear model, then the network, then **boosted trees**
- On tables of high-level features, boosted decision trees are hard to beat: Lecture 20's benchmark paper, on one dataset

Vocabulary

- **module, reproducible, Hillas parameters, Cherenkov light, air shower, gamma vs hadron**
- **manifold, curse of dimensionality, feature hierarchy, hyper-parameter, feature engineering, logistic regression**
- **standardize,** `StandardScaler`**,** `Normalization` **layer,** `adapt`**, non-trainable parameter**
- **activation: sigmoid, tanh, ReLU, leaky ReLU; saturate, vanishing gradient**
- **output layer: sigmoid / softmax / linear; binary and categorical cross-entropy, one-hot, MSE**
- **likelihood, entropy, cross-entropy, KL divergence**
- **gradient descent, learning rate, schedule, mini-batch, epoch, momentum, Adam**
- **under-fitting, over-fitting, learning curve, early stopping, checkpoint, L1 / L2 regularization, dropout**
- **baseline, AUC, boosted decision trees**

What comes next

- Lecture 23: **images**. Convolutional networks, the classic architectures, and **transfer learning**: reusing a network trained on millions of images
- Lecture 24: keeping the hardware busy: processes, threads and parallel computing